

Lecture 11 Q&A

Recursion, printReverse, and DFS: your questions from
the slip

CSCI 235 – Data Structures

Hunter College, Fall 2026

Class 11 Tuesday, October 6, 2026

What You Asked

Thank you for the thoughtful questions on the lecture slip. They fall into six groups:

Theme	Where we answer it
More examples; <code>printReverse</code>	Trace of "abc", forward vs. backward, <code>digitSum</code> , a recipe
DFS again, in general and in more detail	DFS in one slide, then the slip maze step by step
Slip question 3	Trace, decision table, call tree, answers
Left/right/up/down order; unmark and remove?	Does the order matter? What is undone when we backtrack?
BFS vs. DFS; queues; counting cells	Comparison; queue, enqueue, dequeue; size of a circular queue (<code>front_</code> , <code>back_</code>)
<code>size_type</code> ; review- ing	Short slides near the end

More printReverse Examples: Trace "abc"

```
void printReverse(std::string s) {  
    if (s.empty()) { return; }           // base case  
    printReverse(s.substr(1));           // smaller: rest of s  
    std::cout << s[0];                  // combine: then s[0]  
}
```

Call	Calls with	Prints on return	Order
("abc")	"bc"	a	3rd
("bc")	"c"	b	2nd
("c")	" "	c	1st
("")	(base case)	nothing	—

Read it bottom-up

The calls go down: "abc", "bc", "c", "". The printing happens on the way *back up*, so the last character is printed first: cba.

Swap Two Lines: Forward Instead of Backward

Only the position of the print changes.

```
// prints "abc"
void printForward(std::string s) {
    if (s.empty()) { return; }
    std::cout << s[0]; // BEFORE
    printForward(s.substr(1));
}
```

```
// prints "cba"
void printReverse(std::string s) {
    if (s.empty()) { return; }
    printReverse(s.substr(1));
    std::cout << s[0]; // AFTER
}
```

- **Before** the recursive call: work happens on the way *down*, in the original order.
- **After** the recursive call: work happens on the way *back up*, in reverse order.
- Same idea as slip question 1: `show(3)` prints 3 2 1 1 2 3. Try it: what if a print appears both before *and* after the call?

Another Example: Sum of the Digits

`digitSum(472)` should return $4 + 7 + 2 = 13$.

```
int digitSum(int n) {           // n >= 0
    if (n < 10) { return n; } // base case: one digit
    return n % 10 + digitSum(n / 10); // last digit + the rest
}
```

Call	Last digit $n \% 10$	Returns
<code>digitSum(472)</code>	2	$2 + \text{digitSum}(47) = 2 + 11 = 13$
<code>digitSum(47)</code>	7	$7 + \text{digitSum}(4) = 7 + 4 = 11$
<code>digitSum(4)</code>	(base case)	4

Same three parts

Base case: one digit. **Smaller problem:** $n / 10$ drops the last digit.

Combine: add the digit we removed.

A Recipe for Any New Recursion Problem

Ask these four questions, in this order:

- ❶ **What is the smallest version I can answer immediately?**
That is the base case.
- ❷ **What is a slightly smaller version of the same problem?**
(shorter string, smaller number, fewer cells left)
- ❸ **Leap of faith:** if the recursive call already gives the right answer for that smaller problem, what is left to do?
- ❹ **Does every call get closer to the base case?** If not, the stack overflows.

Practice at home

- (1) `countDown(n)` prints n down to 1; then `countUp(n)` prints 1 up to n . Which line moves?
- (2) `power(x, n)` returns x^n using $x^n = x \cdot x^{n-1}$.
- (3) `countChar(s, ch)`: how many times does `ch` appear in `s`?

DFS in General: One Slide

The picture. Explore a maze with chalk and a ball of string:

- Walk as far as you can. Chalk-mark every cell you enter.
- At a **dead end**, follow the string back to the last cell that still has an unexplored direction. That is **backtracking**.
- **Never rub out chalk.** A marked cell is never entered again.

The recursion. Each call asks:

“Starting from this cell, is there a path to the target?”

- Mark the cell visited.
- **Base case:** it is the target: success.
- Otherwise try each neighbor in a fixed order (left, right, up, down). If a neighbor is open and unvisited, call the same function on it.
- If any call succeeds, this call succeeds. If all fail, return failure.

The **call stack** is the ball of string: its frames are the cells on the current route.

Slip Question 3: The Setup

	0	1	2
0	S		
1			
2	T		

$S = (0, 0)$, $T = (2, 0)$. Black cells are walls. Cells are written (row, column).

A call marks its cell visited, then tries its neighbors in the order **left, right, up, down**, recursing into any that is open and unvisited.

- (a) Order in which calls are entered?
- (b) Dead end at (2, 2): which calls return, and where does DFS resume?
- (c) Final route, and why is it not on the first branch?

Orange: current call. Blue: visited, call still active. Gray: visited, returned empty-handed. Green: final route.

Slip Question 3, Step 1 of 9

	0	1	2
0	S		
1			
2	T		

Call stack

helper(0,0)

Answer stack

(empty)

Start: call `helper(0,0)`. Mark it visited. Neighbors: left outside, right **open**. Go there next.

Slip Question 3, Step 2 of 9

	0	1	2
0	S		
1			
2	T		

Call stack

helper(0,1)
helper(0,0)

Answer stack

(empty)

Move right: call `helper(0,1)`. Mark it visited. Neighbors: left visited, right **open**. Go there next.

Slip Question 3, Step 3 of 9

	0	1	2
0	S		
1			
2	T		

Call stack

helper(0,2)
helper(0,1)
helper(0,0)

Answer stack

(empty)

Move right: call `helper(0,2)`. Mark it visited. Neighbors: left visited, right outside, up outside, down **open**. Go there next.

Slip Question 3, Step 4 of 9

	0	1	2
0	S		
1			
2	T		

Call stack

helper(1,2)
helper(0,2)
helper(0,1)
helper(0,0)

Answer stack

(empty)

Move down: call `helper(1,2)`. Mark it visited. Neighbors: left wall, right outside, up visited, down **open**. Go there next.

Slip Question 3, Step 5 of 9

	0	1	2
0	S		
1			
2	T		

Call stack

helper(2,2)
helper(1,2)
helper(0,2)
helper(0,1)
helper(0,0)

Answer stack

(empty)

Move down: call `helper(2,2)`. Mark it visited. Neighbors: left wall, right outside, up visited, down outside. **Dead end**: return.

Slip Question 3, Step 6 of 9

	0	1	2
0	S		
1			
2	T		

Call stack

helper(0,0)

Answer stack

(empty)

helper(2,2), helper(1,2), helper(0,2), helper(0,1) returned empty-handed (backtracking), each leaving the stack. Back in helper(0,0). Remaining neighbors: up outside, down **open**. Go there next.

Slip Question 3, Step 7 of 9

	0	1	2
0	S		
1			
2	T		

Call stack

helper(1,0)
helper(0,0)

Answer stack

(empty)

Move down: call `helper(1,0)`. Mark it visited. Neighbors: left outside, right wall, up visited, down **open**. Go there next.

Slip Question 3, Step 8 of 9

	0	1	2
0	S		
1			
2	T		

Call stack

helper(2,0)
helper(1,0)
helper(0,0)

Answer stack

(2,0)

Move down: call `helper(2,0)`. Mark it visited. It is the **target** (base case): push it on the answer stack and return success.

Slip Question 3, Step 9 of 9

	0	1	2
0	S		
1			
2	T		

Call stack

(no calls left)

Answer stack

(0,0)
(1,0)
(2,0)

Success unwinds the call stack: each frame sees a non-empty answer stack, pushes its own cell and returns. The route is $(0,0) \rightarrow (1,0) \rightarrow (2,0)$. The cells on the first branch were *visited* but are not on the route.

Slip Question 3: The Answers

(a) Entered: $(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (1, 0), (2, 0)$.

(b) At $(2, 2)$ the left neighbor is a wall, the up neighbor is visited, and the right and down neighbors are outside, so the call returns empty-handed to $(1, 2)$. Then $(1, 2)$ returns to $(0, 2)$, $(0, 2)$ to $(0, 1)$, and $(0, 1)$ to $(0, 0)$: none has another open, unvisited neighbor. At $(0, 0)$ the loop continues: up is outside, and down, $(1, 0)$, is open and unvisited, so DFS resumes there.

(c) Route: $(0, 0) \rightarrow (1, 0) \rightarrow (2, 0)$. DFS tries *right* before *down*, and the whole right-hand branch ends in a dead end. None of its cells are pushed on the answer stack: they were *visited*, but they are not on the route.

Key distinction

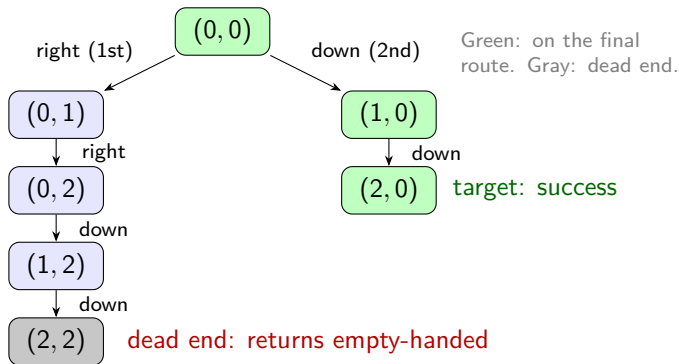
Visited means “a call entered this cell”. **On the route** means “a call below it reached the target”. Seven cells were visited; three are on the route.

Each Call's Choice Between Its Neighbors

Call	left	right	up	down	Outcome
(0,0)	outside	open: call (0,1)	outside	open: call (1,0)	success through down
(0,1)	visited	open: call (0,2)	outside	wall	fails
(0,2)	visited	outside	outside	open: call (1,2)	fails
(1,2)	wall	outside	visited	open: call (2,2)	fails
(2,2)	wall	outside	visited	outside	dead end , fails
(1,0)	outside	wall	visited	open: call (2,0)	success
(2,0)	target: base case				success

- A neighbor is skipped for exactly three reasons: **outside** the grid, a **wall**, or **already visited**.
- (0,1)'s only open neighbor leads nowhere, so it fails too. A call fails when every neighbor was skipped or failed.
- Only (0,0) has *two* calls to make: right (failed), then down (succeeded).

The Same Search as a Call Tree



Each node is one call (one stack frame). Going down an edge is a call; coming back up is a return. The left branch is explored completely first, and *only then* does (0,0) try its second choice.

Does the Neighbor Order Matter?

Question: on slip question 3, must I try left and right before down, or can I go straight down?

As the slip says

Left, right, up, down: 7 calls, $(0,0), \dots, (2,2), (1,0), (2,0)$. Route $(0,0) \rightarrow (1,0) \rightarrow (2,0)$.

Down first (not asked)

3 calls, $(0,0), (1,0), (2,0)$. Same route here.

- Any fixed order is a legitimate DFS: it finds a path whenever one exists. Different orders can enter cells in a different order and may find a different route.
- So the order is **part of the question**. Follow the order that the problem or project states. Slip question 3 says left, right, up, down, so right is tried before down and the answer to (a) is the 7-cell list.
- Project 2 also fixes this order, so that every correct solution produces the same route and the same output.

When We Backtrack: What Is Undone?

Question: do we unmark the cells we used? Do we remove them from stack memory?

Thing	Result	Why
Visited mark of the cell	Stays marked	For this DFS, once a cell has been explored we do not need to explore it again. Un-marking could make the search explore the same cells and dead ends repeatedly.
The call's stack frame	Removed	Returning from a call pops its frame automatically. You do not write any code for this.
Answer stack	Unchanged	Only calls that were followed by success push their cell. Dead-end cells never push.

In one sentence

“Backtracking” is just **returning** from a call that failed; the visited marks remain, and the frames disappear by themselves.

BFS vs. DFS

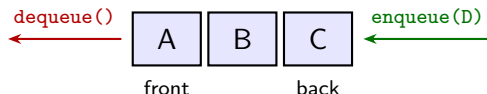
	DFS	BFS
Idea	Go as deep as possible along one branch, then back up	Explore all cells at distance 1, then 2, then 3, ...
Container	Stack (the call stack, or an explicit one)	Queue
Route found	The first route it finds: not necessarily the shortest	A route with the fewest cells
Slip maze	Enters 7 cells before reaching T	Reaches T after looking at 5 cells
Project 2 example (4×4)	Route of 9 cells	Route of 7 cells

Next class: we write BFS with an explicit queue and compare the two on the same maze.

What Is a Queue? (and “enqueue”)

I read the question as “what is a queue (enqueue / dequeue)?” If you meant something else, ask me in class.

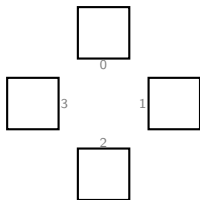
A **queue** is a line at a store: the first person to join is the first person served. We say **FIFO**: first in, first out.



- **enqueue(x)**: add x at the back. **dequeue()**: remove and return the item at the front.
- A **stack** is the opposite: last in, first out (a pile of plates).
- Why we care today: DFS uses a stack (the call stack); BFS next class uses a **queue**. Class 10 covered queues in detail.

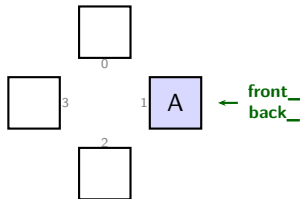
Counting Elements: Empty and One Element

Empty



$\text{front_} = \text{back_} = -1$
 $\text{size} = 0$

One element

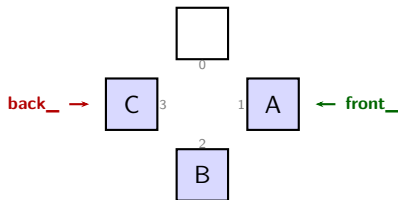


$\text{front_} = \text{back_} = 1$
 $\text{size} = 1 - 1 + 1 = 1$

CAPACITY = 4. front_ is the index of the *first* element and back_ the index of the *last*. An empty queue has no first or last element, so both are -1 and the size is 0. With exactly one element, both indices name the *same* cell, and $\text{back_} - \text{front_} + 1 = 1$.

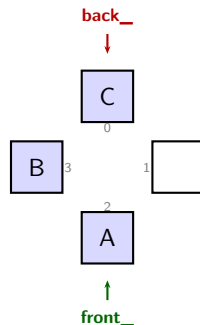
Counting Elements: Not Wrapped and Wrapped

Not wrapped



$$\text{front_} = 1, \text{back_} = 3 \\ 3 - 1 + 1 = 3$$

Wrapped

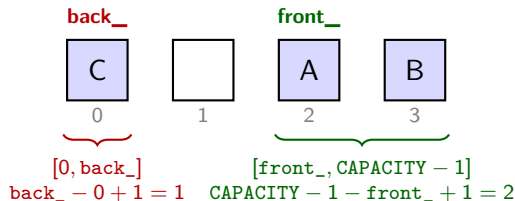


$$\text{front_} = 2, \text{back_} = 0 \\ 4 - (2 - 0 - 1) = 3$$

Not wrapped ($\text{back_} \geq \text{front_}$): cells $\text{front_} \dots \text{back_}$ are used:
 $\text{back_} - \text{front_} + 1$. **Wrapped** ($\text{back_} < \text{front_}$): the queue runs past the end and continues at cell 0. Only the $\text{front_} - \text{back_} - 1$ cells between `back_` and `front_` are empty (here cell 1), so $\text{size} = \text{CAPACITY} - (\text{front_} - \text{back_} - 1)$.

Wrapped Queue: A Second Way to Count

Cut the wrapped queue into two pieces that are each *not* wrapped (CAPACITY = 4, front_ = 2, back_ = 0):



Add the two pieces

$(\text{back_} + 1) + (\text{CAPACITY} - \text{front_})$. Here: $(0 + 1) + (4 - 2) = 3$.

Same as the first formula:

$\text{CAPACITY} - (\text{front_} - \text{back_} - 1) = \text{CAPACITY} - \text{front_} + \text{back_} + 1$. Count the *gap* and subtract, or count the *two pieces* and add.

Counting Elements: The Rule and the Code

Case	Test	Size
Empty	<code>front_ == -1</code>	0
Not wrapped	<code>back_ >= front_</code>	<code>back_ - front_ + 1</code>
Wrapped	<code>back_ < front_</code>	<code>CAPACITY - (front_ - back_ - 1)</code>

Examples with `CAPACITY = 4`, as $(\text{front}_-, \text{back}_-)$: $(-1, -1) \rightarrow 0$, $(2, 2) \rightarrow 1$, $(1, 3) \rightarrow 3$, $(2, 0) \rightarrow 3$, $(1, 0) \rightarrow 4$.

```
int size() const {
    if (front_ == -1) { return 0; }           // empty
    if (back_ >= front_) { return back_ - front_ + 1; } // not wrapped
    return CAPACITY - (front_ - back_ - 1);   // wrapped
    // same value: back_ + 1 + (CAPACITY - front_)
}
```

- Why subtract? The array has `CAPACITY` cells. In the wrapped case, every cell is either in the queue or in the gap between `back_` and `front_`.
- Check the full wrapped queue $(1, 0)$: $\text{gap} = 1 - 0 - 1 = 0$, so $\text{size} = 4 - 0 = 4$.

What Is `std::string::size_type`?

It is the **unsigned integer type** that `std::string` uses for lengths and positions. `s.length()` returns it. On a typical 64-bit system it is the same as `std::size_t` (an 8-byte unsigned integer).

```
void printReverse(const std::string& s,
                  std::string::size_type i) {
    if (i == s.length()) { return; } // same type on both sides
    printReverse(s, i + 1);
    std::cout << s[i];
}
```

- We use it for `i` so that `i == s.length()` compares two values of the *same* kind (no signed/unsigned warning).
- You may write `int i` in simple code, but the compiler may warn when you compare an `int` with `s.length()`.
- **Careful:** unsigned values never go below 0. For an empty string, `s.length() - 1` is not `-1`: it wraps to a huge number (18446744073709551615 on a typical 64-bit system).

Reviewing Arrays, Stacks, and Queues

Reviewing is a good plan: recursion builds directly on them.

- **Stack** (Class 9): the **call stack** behaves like a stack. A call pushes a frame; a return pops it; the newest call finishes first.
- **Queue** (Class 10): BFS next class uses a queue, so the circular-buffer ideas return.
- **Where to review:** the Class 9 and Class 10 slides, the Class 10 Q&A deck, the Class 10 and Class 11 flashcards, and the Queue and Recursion Card Quest games on the schedule page.

A 10-minute routine

Do one card-game round, then redo slip questions 1 and 3 from memory, then compare with the key.

Take-Aways

- 1 Every recursion needs a **base case**, a **smaller problem**, and a **combine** step. Printing *before* the call goes forward; printing *after* it goes backward.
- 2 DFS = “is there a path from here?” Mark the cell, try the neighbors in the stated order, and return success or failure.
- 3 Backtracking is just **returning**: frames pop, **visited marks stay**.
- 4 **Visited** is not the same as **on the route**.
- 5 DFS uses a **stack**; BFS (next class) uses a **queue**.

Keep asking questions on the slips: they shape the next class.